University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Concession Test, 2023

Answer all questions.
Approximate marks per question are shown in brackets.

Time: 45 minutes + 5 minutes reading time **Marks: 35**

Section 1: ASD1 [9 marks]

Question 1. **[3]**

Consider the concept of major and minor releases in software. Explain how the two releases are different from one another, and what types of changes (and bug fixes) might go in each. Where do software architectures factor into the differences?

A minor release is a small piece of functionality which would not normally require a change management process and should have minimal bug fixes. Minor releases are further unlikely to affect the systems architecture. Major releases add a large set of features which would normally be agreed upon during a change management process and might demand changes to the current system architecture.

+1 minor releases include mostly bug fixes and small features

+1 major releases include bug fixes, especially those that may require api adjustments or more extensive changes to the software. However mostly they are recognizable by the increased scale of new features

+1 major can entail software arch changes, and usually require more extensive testing, minor should not have arch changes, and usually occur within an architectural component, small changes that are not likely to cause other bugs can be introduced in minor releases.

Question 2. **[6]**

A common risk in software engineering projects is missed deadlines due to poor estimates or unanticipated delays. Define the 3Ms of risk management and use them to explain how you might address this risk. State any assumptions.

Mitigation: How to avoid or reduce the risk, or assume the risk and accept and control it. [1] You can mitigate risk by having an experienced team (better estimates) and using an agile approach that prioritizes important features first. Any appropriate answer is fine. Wrong answers not related to mitigation are not fine. [1]

Monitoring: Track factors to enable us to determine if the risk is becoming more or less likely. [1] Tools like a scrum burndown chart can help track whether estimates are

matching reality. Tracking progress and daily checkins help make this more transparent. [1]

Management: Make contingency plans to deal with the risk if it becomes a reality. [2]

Section 2: ASD2 [9 marks]

Question 3. **[9 marks]**

a) Refactor class Worker below so it adheres to the Single Responsibility Principle (SRP) by **naming the classes** you would replace it with, **and naming only the methods in each** class. Briefly state **why** you created the classes in your revised structure. [4]

Worker
- workerID:String - name:String - wage:int
+ getID():String + setID(…) + getName():String + setName(…) + getWage():int + setWage(…) + checkPromotable(…):Boolean + calcTax(…):Double

remove checkPromotable() and calcTax() from Worker ✓

make class Promotion with method checkPromotable(Worker w): Boolean ✓

make class Taxation with method calcTax(Worker w): Double ✓

because don't want to change Workers every time promotion criteria/taxation rules change ✓

b) Name and briefly describe 5 architecture patterns (1 sentence each). [5]

layered, MVC, repository, client-server, pipe-and-filter (1/2 mark for name, 1/2 for description)

Section 3. Theory of Algorithms (TOA) [9 marks]

Question 4 **[6]**

a) Which statement(s), among the following, is FALSE about breadth-first search when it is used on graph-based problems [3]

(i) A queue must be used to keep track of adjacent nodes and the order in which they need to be visited.
(ii) The algorithm's efficiency for adjacency lists is BigTheta(|V|) where represents the vertices/nodes
(iii) There are two orderings of the nodes that can be obtained via the algorithm
(iv) Topological sorting cannot be achieved using BFS since only Depth-First Search is appropriate.
(v) It can be used to detect cycles in directed graphs but not undirected graphs.

(ii, iii, iv, v) are all false. 3 marks if the answer is FULLY correct, 1 mark if the answer is partially correct.

b) **Fake-Coin problem:** consider the following algorithm for finding a single coin that weighs less than all others from an array of n coins where all others have the same weight and answer the question that follows. [2]

```
ALGORITHM findFakeCoin(coins):
    for i in range(1,n-2) do
        left = coins[i-1]
        mid = coins[i]
        right = coins[i+1]
        if left + mid < mid + right do
            return min(left, mid)
        else left + mid > mid + right do
            return min(mid, right)
```

Is the algorithm an example of a divide-and-conquer strategy? Explain your answer.

No. The input is never split into m equal sub-problems. Instead, the for-loop iterates over n-2 of the items and compares the current coin to its left and right hand side.

c) Consider the text "This is a problem" and the pattern "prob". How many unsuccessful alignments of the text and pattern will be conducted by Horspool's algorithm before it finds the pattern? [1]

"prob" compared to "This" and is unsuccessful
"prob" compared to " is " and is unsuccessful
"prob" compared to "a pr" and is unsuccessful
"prob" compared to "prob" but it is successful
ANS: Three

Question 5 **[3 marks]**

Consider the following scenario and answer the question that follows:

You are given the following messages and each message's associated measure of information content.

Message 1 = ('a', 2)
Message 2 = ('bb', 4)
Message 3 = ('bca', 16)

The cost of sending a single character is 1 cent and you want to transmit messages with the most amount of information (measured in bits).

Write down the recurrence relation, if you are asked to use Dynamic Programming, to calculate the maximum amount of information you can transmit if you can only transmit at most n cents worth of information.

Let i $(0 \leq i \leq 3)$ represent the transfer of i messages and j $(0 \leq j \leq n)$ represent the amount of information being sent. Furthermore, let the cost of sending i messages be represented by $cost_i$ and the information amount associated with sending i messages be $info_i$ then the optimal solution can calculated as follows:

$$C(i,j) = \begin{cases} max\{C(i-i,j), info_i + C(i-1, j-cost_i)\} & \text{if } j \geq cost_i \\ C(i-i,j) & \text{if } j < cost_i \end{cases}$$

Where $C(0,j) = 0$ for $j \geq 0$ and $C(i,0) = 0$ for $i \geq 0$

(1 mark for each condition in the equation, 1 mark for specifying what the variables are)

Section 4. Theory of Computation (TOC) [8 Marks]

Question 9 **[1]**

Consider the following Turing machine with states q, p, and r, where q is the start state and r the final state, Γ = {0, 1, □} (where the box "□" indicates the blank symbol), Σ = {0,1}, and the following transition functions:

State	Tape Symbol	Move
q	0	(q,0,R)
q	1	(p,0,R)

q	□	(q, □,R)
p	0	(q,0,L)
p	1	(r,1,R)
p	□	(p,0,R)

Describe a property of the input strings that makes this TM accept by final state. Then select the string from the options listed which make this TM accept.

a. 01100
b. 010□1□1
c. 0001
d. 1010

Answer: A

Question 10 **[3]**

Suppose we have the following four languages and all we know about them is the following:

- A is Recursive
- B Recursively Enumerable but not Recursive.
- C is Recursively Enumerable
- D is non-Recursively Enumerable.

Select all the correct statements from the options below:

a. There definitely exists a function R such that C reduces to A.
b. There cannot be a reduction from D to B, because else it would prove that B is non-Recursively Enumerable, which it is not.
c. If $\overline{B}$ (complement of B) is also Recursively Enumerable, then both $\overline{B}$ and B are recursive.
d. Either concatenating A and B or taking the union of A and B may give us C.

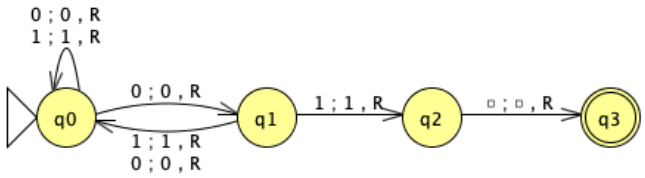
Answer: B and C. 1.5 mark for each.

Question 11 [4 Marks]

Design a Turing Machine that accepts all strings that have 01 as its last two characters.

Hint: first describe in short statements the three component tasks that the TM needs to accomplish. Then draw the state diagram that accomplishes that.

Answer: an NTM is the easiest:



Marking guide: **deduct 0.5** for not indicating start state (an incoming arrow, -->), and **deduct 0.5** for not indicating final state (double lined circle). **A state diagram as above gains the 4 marks.** If they write all the transition functions as functions, then **also** give the full marks.

If they draw a TM that accept only 01 (i.e., no loop on q0 and no transition back from q1 to q0), then give **1 mark**.

If they can describe how their TM would solve it even if their state diagram is wrong or is not present, Give **2 marks** when says something along the line of:

1) It steps over the 0s and 1s for any number of times
2) At some point, it guesses that it will be the last 0 of the input;
3) Then it guesses that the next symbol is the final 1;
4) When it sees a blank afterward, it goes to the final state.

With **0.5** for each sub-task. Marking note: talking of *traces* instead of *guesses* is fine as well, like “at least one trace”.

-END-